
Lattices and Fixpoints

Lecture 3 — Static Analysis by Abstract Interpretation

David Pichardie

Fall 2026

Lecture 2 defined the collecting semantics but left three questions open:

- $\llbracket \text{while } b \text{ do } c \rrbracket S = \llbracket \neg b \rrbracket (\text{lfp } \Phi_S)$ — but what *is* lfp , and why does it exist?
- the iterates $X_0 \subseteq X_1 \subseteq \dots$ stabilised in every example — by luck, or always?
- $\llbracket \cdot \rrbracket$ was *declared* to be the right object — never checked against $\Downarrow$.

Other questions will arise:

- an abstract interpreter will use abstract operations $\llbracket \cdot \rrbracket^\sharp$: what will play the roles of $\subseteq$ and $\cup$?
- which more efficient fixpoint-solving techniques are available?

Orders and lattices

Fixpoints: Knaster–Tarski and Park

Soundness of the collecting semantics

Existence of the abstract interpreter

Fixpoint computation on equation systems

§ 1

Orders and lattices

Definition 1 (Partial order)

A *partial order* on a set L is a relation $\sqsubseteq$ that is **reflexive** ($x \sqsubseteq x$), **transitive** and **antisymmetric** ($x \sqsubseteq y \wedge y \sqsubseteq x \Rightarrow x = y$). “Partial”: some pairs may be incomparable.

How to read $\sqsubseteq$ in this course

$x \sqsubseteq y$ means “ x is a **less coarse approximation** than y ”.

The first partial order we meet is $\mathcal{P}(\Sigma)$, equipped with $\sqsubseteq$:

- $\emptyset$ means that a program point is unreachable (**strong information**);
- Σ means that any state may be reachable (**weak information**).

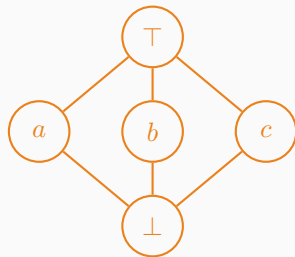
Custom partial orders and Hasse diagrams

On a finite domain, we can define a partial order by drawing a *Hasse diagram*.

Rules and conventions

- edges have no arrowheads: an upward path from x to y represents $x \sqsubseteq y$;
- reflexive and transitive edges are omitted.

Thus a , b , and c are pairwise incomparable, while $\perp \sqsubseteq a \sqsubseteq \top$ implies $\perp \sqsubseteq \top$ without a direct edge.



Upper and lower bounds

Let $S \subseteq L$. An *upper bound* of S is a u with $x \sqsubseteq u$ for all $x \in S$; the *least upper bound* $\sqcup S$ is an upper bound below every other one. Dually the *greatest lower bound* $\sqcap S$. Antisymmetry makes them *unique when they exist*.

We write $x \sqcup y = \sqcup\{x, y\}$ and $x \sqcap y = \sqcap\{x, y\}$.

Example in the preceding Hasse diagram

The upper bounds of $\{a, b\}$ are exactly $\{\top\}$, so $a \sqcup b = \top$. Its lower bounds are exactly $\{\perp\}$, so $a \sqcap b = \perp$.

Definition 2 (Lattice)

A partially ordered set $(L, \sqsubseteq)$ is a *lattice* when every pair $x, y \in L$ has

- a least upper bound $x \sqcup y$, called their *join*;
- a greatest lower bound $x \sqcap y$, called their *meet*.

Example

The usual order on $\mathbb{Z}$ is total: every two integers are comparable. Moreover, every pair has a minimum and a maximum, so $(\mathbb{Z}, \leq)$ is a lattice:

$$x \sqcup y = \max(x, y), \quad x \sqcap y = \min(x, y).$$

Definition 3 (Complete lattice)

$(L, \sqsubseteq)$ is a *complete lattice* when *every* subset $S \subseteq L$ has a least upper bound $\bigsqcup S$.

A lattice only provides bounds for pairs. For example, $(\mathbb{Z}, \leq)$ is a lattice but is not complete: $\mathbb{Z}$ itself has no upper bound.

The one we care about today

$(\mathcal{P}(X), \subseteq)$ is a complete lattice, with $\bigsqcup = \bigcup$. Taking $X = \Sigma$ gives the concrete domain of the whole course.

Intuitively, $\bigsqcup$ acts as a limit operator: completeness means that these limits remain inside L .

Exercise — three things come for free

Let $(L, \sqsubseteq)$ be a complete lattice. Using only least upper bounds, prove:

1. L has a greatest element $\top = \bigsqcup L$;
2. L has a least element $\perp = \bigsqcup \emptyset$;
3. every $S \subseteq L$ has a greatest lower bound, given by $\bigsqcap S = \bigsqcup \{y \in L \mid \forall x \in S, y \sqsubseteq x\}$.

Exercise — three things come for free

Let $(L, \sqsubseteq)$ be a complete lattice. Using only least upper bounds, prove:

1. L has a greatest element $\top = \bigsqcup L$;
2. L has a least element $\perp = \bigsqcup \emptyset$;
3. every $S \subseteq L$ has a greatest lower bound, given by $\bigsqcap S = \bigsqcup \{y \in L \mid \forall x \in S, y \sqsubseteq x\}$.

Solution

$$\forall x \in L, \quad x \sqsubseteq \bigsqcup L,$$

$$\text{hence } \top = \bigsqcup L,$$

Exercise — three things come for free

Let $(L, \sqsubseteq)$ be a complete lattice. Using only least upper bounds, prove:

1. L has a greatest element $\top = \bigsqcup L$;
2. L has a least element $\perp = \bigsqcup \emptyset$;
3. every $S \subseteq L$ has a greatest lower bound, given by $\bigsqcap S = \bigsqcup \{y \in L \mid \forall x \in S, y \sqsubseteq x\}$.

Solution

$$\begin{array}{ll} \forall x \in L, & x \sqsubseteq \bigsqcup L, \\ \forall x \in L, & \underbrace{\forall y \in \emptyset, y \sqsubseteq x}_{\text{vacuously true}} \implies \bigsqcup \emptyset \sqsubseteq x, \end{array} \qquad \begin{array}{l} \text{hence } \top = \bigsqcup L, \\ \text{hence } \perp = \bigsqcup \emptyset, \end{array}$$

Exercise — three things come for free

Let $(L, \sqsubseteq)$ be a complete lattice. Using only least upper bounds, prove:

1. L has a greatest element $\top = \bigsqcup L$;
2. L has a least element $\perp = \bigsqcup \emptyset$;
3. every $S \subseteq L$ has a greatest lower bound, given by $\bigsqcap S = \bigsqcup \{y \in L \mid \forall x \in S, y \sqsubseteq x\}$.

Solution

$$\begin{aligned} \forall x \in L, \quad x &\sqsubseteq \bigsqcup L, & \text{hence } \top &= \bigsqcup L, \\ \forall x \in L, \quad \underbrace{\forall y \in \emptyset, y \sqsubseteq x}_{\text{vacuously true}} &\implies \bigsqcup \emptyset \sqsubseteq x, & \text{hence } \perp &= \bigsqcup \emptyset, \\ B_S &\triangleq \{y \in L \mid \forall x \in S, y \sqsubseteq x\}, & m &\triangleq \bigsqcup B_S, \\ x \in S &\implies \forall y \in B_S, y \sqsubseteq x \implies m \sqsubseteq x, & m &\text{ is a lower bound,} \\ \forall x \in S, z \sqsubseteq x &\implies z \in B_S \implies z \sqsubseteq m, & m &\text{ is the greatest one,} \\ \boxed{\bigsqcap S = m}. \end{aligned}$$

Product — componentwise

If $L_1, \dots, L_n$ are complete lattices, so is $L_1 \times \dots \times L_n$ under $(x_1, \dots, x_n) \sqsubseteq (y_1, \dots, y_n) \iff \forall i, x_i \sqsubseteq_i y_i$, with $\sqcup$ taken componentwise.

Function space — pointwise

If L is a complete lattice and X any set, then $X \rightarrow L$ is a complete lattice under $f \sqsubseteq g \iff \forall x, f(x) \sqsubseteq g(x)$, with $(\sqcup F)(x) = \sqcup \{f(x) \mid f \in F\}$.

Chains and the ascending chain condition

A *chain* is a totally ordered subset; an *ascending chain* is a sequence $x_0 \sqsubseteq x_1 \sqsubseteq x_2 \sqsubseteq \dots$.

Definition 4 (ACC, height)

L satisfies the *ascending chain condition* when every ascending chain is eventually constant — equivalently, no infinite *strictly* increasing chain. The *height* of L is the supremum of the lengths of its strict chains; it may be infinite and need not be attained by any chain.

- $\mathcal{P}(X)$ has height $|X|$: it satisfies the ACC *exactly when X is finite*.
- So $\mathcal{P}(\Sigma)$ does *not* — witness $\{0\} \subset \{0, 1\} \subset \{0, 1, 2\} \subset \dots$.
- The ACC does *not* imply finite height: finite strict chains may have arbitrarily large lengths.

Definition 5

$F : L \rightarrow L$ is *monotone* when $x \sqsubseteq y \Rightarrow F(x) \sqsubseteq F(y)$, and *continuous* when it preserves the lubs of ascending chains: $F(\bigsqcup_k x_k) = \bigsqcup_k F(x_k)$.

Continuous $\Rightarrow$ monotone; the converse fails in general.

Theorem 6

If c contains no loop, then $\llbracket c \rrbracket$ is continuous — and therefore monotone.

This is also true when c contains loops, but proving the loop case requires the fixpoint theorems. We will return to it afterwards.

Continuity of loop-free commands (1/2)

Let $U = \bigcup_i S_i$. We prove the stronger equality $\llbracket c \rrbracket U = \bigcup_i \llbracket c \rrbracket S_i$ by structural induction on c .

$$\llbracket \text{skip} \rrbracket U = U = \bigcup_i \llbracket \text{skip} \rrbracket S_i,$$

$$\begin{aligned}\llbracket x := a \rrbracket U &= \{ \sigma[x \mapsto \llbracket a \rrbracket \sigma] \mid \sigma \in U \} \\ &= \{ \sigma[x \mapsto \llbracket a \rrbracket \sigma] \mid \exists i, \sigma \in S_i \} \\ &= \bigcup_i \llbracket x := a \rrbracket S_i,\end{aligned}$$

$$\llbracket b \rrbracket U = U \cap \{b\} = \bigcup_i \llbracket b \rrbracket S_i.$$

Continuity of loop-free commands (2/2)

Let $U = \bigcup_i S_i$. Using the induction hypotheses:

$$\begin{aligned}\llbracket c_1 ; c_2 \rrbracket U &= \llbracket c_2 \rrbracket (\llbracket c_1 \rrbracket U) \\ &= \llbracket c_2 \rrbracket \left(\bigcup_i \llbracket c_1 \rrbracket S_i \right) \\ &= \bigcup_i \llbracket c_2 \rrbracket (\llbracket c_1 \rrbracket S_i) = \bigcup_i \llbracket c_1 ; c_2 \rrbracket S_i,\end{aligned}$$

$$\begin{aligned}\llbracket \text{if } b \text{ then } c_1 \text{ else } c_2 \rrbracket U &= \llbracket c_1 \rrbracket (\llbracket b \rrbracket U) \cup \llbracket c_2 \rrbracket (\llbracket \neg b \rrbracket U) \\ &= \bigcup_i \llbracket c_1 \rrbracket (\llbracket b \rrbracket S_i) \cup \bigcup_i \llbracket c_2 \rrbracket (\llbracket \neg b \rrbracket S_i) \\ &= \bigcup_i (\llbracket c_1 \rrbracket (\llbracket b \rrbracket S_i) \cup \llbracket c_2 \rrbracket (\llbracket \neg b \rrbracket S_i)) = \bigcup_i \llbracket \text{if } b \text{ then } c_1 \text{ else } c_2 \rrbracket S_i.\end{aligned}$$

§2

Fixpoints: Knaster–Tarski and Park

The vocabulary of fixpoints

Let $F : L \rightarrow L$. We write $\text{Fix}(F) = \{x \mid F(x) = x\}$, and $\text{lfp}F$ (resp. $\text{gfp}F$) for its least (resp. greatest) element when it exists.

Two one-sided variants matter more than fixpoints themselves:

$$F(x) \sqsubseteq x \quad \text{and} \quad x \sqsubseteq F(x).$$

A terminology clash in the literature

A point with $F(x) \sqsubseteq x$ is a *pre-fixpoint* for Tarski and Winskel, and a *post-fixpoint* for Cousot and Miné. Only the inequality is unambiguous, so prefer writing $F(x) \sqsubseteq x$ out.

Reading: $F(x) \sqsubseteq x$ says that x is *closed* under one step of F , that is, an *inductive invariant*. This is what Park induction exploits.

Theorem 7 (Knaster–Tarski)

Let L be a complete lattice and $F : L \rightarrow L$ *monotone*. Then $\text{Fix}(F)$ is non-empty, and

$$\text{lfp}F = \bigcap \{ x \in L \mid F(x) \sqsubseteq x \}.$$

Dually, $\text{gfp}F = \bigcup \{ x \mid x \sqsubseteq F(x) \}$.

Proof. Put $A = \{ x \mid F(x) \sqsubseteq x \}$ and $a = \bigcap A$ (it exists: L is complete).

- For every $x \in A$: $a \sqsubseteq x$, so $F(a) \sqsubseteq F(x) \sqsubseteq x$ by monotonicity. Thus $F(a)$ is a lower bound of A , giving $F(a) \sqsubseteq a$ — so $a \in A$.
- Applying F to $F(a) \sqsubseteq a$ gives $F(F(a)) \sqsubseteq F(a)$, i.e. $F(a) \in A$, hence $a \sqsubseteq F(a)$.

By antisymmetry $F(a) = a$. And every fixpoint lies in A , so a is below all of them. ■

In practice, this greatest lower bound is not a practical computational tool. We will characterize least fixpoints differently for computation.

Corollary 8 (Park)

If F is monotone on a complete lattice and $F(A) \sqsubseteq A$, then $\text{lfp} F \sqsubseteq A$.

Immediate from Theorem 7: A belongs to the set whose greatest lower bound is $\text{lfp} F$.

A proof by induction

Think of F as the induction step. To prove that every element generated from $\perp$ is below A :

- the base case $\perp \sqsubseteq A$ holds automatically;
- the induction step is exactly $F(A) \sqsubseteq A$.

Park induction then concludes $\text{lfp} F \sqsubseteq A$.

An inductive definition is a least fixpoint in a powerset lattice.

Knaster–Tarski says $\text{lfp} F$ *exists*; it does not say how to get it. Iterating from the bottom does:

$$\perp \sqsubseteq F(\perp) \sqsubseteq F^2(\perp) \sqsubseteq \dots$$

(increasing, by monotonicity and $\perp \sqsubseteq F(\perp)$).

Theorem 9 (Kleene)

If F is *continuous*, then $\text{lfp} F = \bigsqcup_{k \geq 0} F^k(\perp)$.

The sequence approaches the least fixpoint, but need not reach it after finitely many iterations — unless the lattice satisfies the *ACC*.

Proof

Write $a = \bigsqcup_k F^k(\perp)$. Continuity gives $F(a) = \bigsqcup_k F^{k+1}(\perp) = a$; and $F^k(\perp) \sqsubseteq x$ for every x with $F(x) \sqsubseteq x$ (induction on k), so $a \sqsubseteq \text{lfp} F$. Conversely, a is a fixpoint, hence $\text{lfp} F \sqsubseteq a$. Therefore $a = \text{lfp} F$.

Exercise — a Kleene sequence that never stabilises

On $(\mathcal{P}(\mathbb{N}), \subseteq)$, consider

$$F(X) \triangleq \{0\} \cup \{n + 1 \mid n \in X\}.$$

1. Compute $F^0(\emptyset), \dots, F^4(\emptyset)$.
2. Does this sequence stabilise after finitely many steps?
3. Compute its least upper bound and deduce $\text{lfp} F$.

Exercise — a Kleene sequence that never stabilises

On $(\mathcal{P}(\mathbb{N}), \subseteq)$, consider

$$F(X) \triangleq \{0\} \cup \{n + 1 \mid n \in X\}.$$

1. Compute $F^0(\emptyset), \dots, F^4(\emptyset)$.
2. Does this sequence stabilise after finitely many steps?
3. Compute its least upper bound and deduce $\text{lfp} F$.

Solution

For every $k \geq 0$,

$$F^k(\emptyset) = \{0, \dots, k - 1\},$$

with $F^0(\emptyset) = \emptyset$. Every inclusion is strict, so the sequence never stabilises. Nevertheless,

$$\bigsqcup_{k \geq 0} F^k(\emptyset) = \bigcup_{k \geq 0} \{0, \dots, k - 1\} = \mathbb{N} = \text{lfp} F.$$

This is exactly why Kleene uses a limit rather than a finite iterate.

Back to the collecting semantics of while

For a loop `while  $b$  do  $c$` entered from a set of states S , let

$$T \triangleq \llbracket c \rrbracket \circ \llbracket b \rrbracket$$

be one iteration of the loop, and define

$$\Phi_S(X) \triangleq S \cup T(X).$$

Do not assume yet that $\text{lfp}\Phi_S$ exists. Construct the candidate

$$H_S \triangleq \bigcup_{i \geq 0} T^i(S), \quad \llbracket \text{while } b \text{ do } c \rrbracket S \triangleq \llbracket \neg b \rrbracket (H_S).$$

H_S collects the states that reach the loop head after any finite number of iterations. We now prove

$$\boxed{H_S = \text{lfp}\Phi_S}.$$

H_S is a fixpoint of Φ_S

The structural induction establishes simultaneously that $\llbracket c \rrbracket$ is well-defined and preserves unions. For the loop body, the induction hypothesis gives both properties for T ; in particular, T is monotone.

Since T preserves unions,

$$\begin{aligned}\Phi_S(H_S) &= S \cup T \left(\bigcup_{i \geq 0} T^i(S) \right) \\ &= S \cup \bigcup_{i \geq 0} T^{i+1}(S) \\ &= \bigcup_{i \geq 0} T^i(S) \\ &= H_S.\end{aligned}$$

Thus H_S is a fixpoint.

H_S is the least fixpoint of Φ_S

Let X be a fixpoint of Φ_S . Since $\Phi_S(X) = S \cup T(X) = X$, we have

$$S \subseteq X \quad \text{and} \quad T(X) \subseteq X.$$

We prove $T^i(S) \subseteq X$ by induction on i :

- $i = 0$: $T^0(S) = S \subseteq X$;
- $i + 1$: from $T^i(S) \subseteq X$, monotonicity gives

$$T^{i+1}(S) \subseteq T(X) \subseteq X.$$

Therefore

$$H_S = \bigcup_{i \geq 0} T^i(S) \subseteq X.$$

Thus H_S is below every fixpoint of Φ_S . Since H_S is itself a fixpoint, $H_S = \text{lfp} \Phi_S$.

Continuity of $S \mapsto \text{lfp}\Phi_S$ — because it preserves unions

For an arbitrary family $(S_j)_j$, preservation of unions by T gives preservation by every iterate T^i .
Hence

$$\begin{aligned} H_{\bigcup_j S_j} &= \bigcup_{i \geq 0} T^i \left(\bigcup_j S_j \right) \\ &= \bigcup_{i \geq 0} \bigcup_j T^i(S_j) \\ &= \bigcup_j \bigcup_{i \geq 0} T^i(S_j) \\ &= \bigcup_j H_{S_j}. \end{aligned}$$

Therefore

$$S \longmapsto H_S = \text{lfp}\Phi_S$$

preserves arbitrary unions, and is in particular continuous.

The loop semantics is well-defined

The exit filter preserves unions. Using the previous result,

$$\begin{aligned}\llbracket \text{while } b \text{ do } c \rrbracket \left(\bigcup_j S_j \right) &= \llbracket \neg b \rrbracket \left(H_{\bigcup_j S_j} \right) \\ &= \llbracket \neg b \rrbracket \left(\bigcup_j H_{S_j} \right) \\ &= \bigcup_j \llbracket \neg b \rrbracket (H_{S_j}) = \bigcup_j \llbracket \text{while } b \text{ do } c \rrbracket S_j.\end{aligned}$$

Main conclusion — well-definedness

For every command c , $\llbracket c \rrbracket$ is well-defined. In particular, in the loop case the required least fixpoint exists because $H_S = \text{lfp } \Phi_S$.

Additional result. The same structural induction shows that every $\llbracket c \rrbracket$ preserves unions; hence it is continuous, and therefore monotone.

§3

Soundness of the collecting semantics

Relating $\llbracket \cdot \rrbracket$ to the operational semantics

Lecture 2 *defined* $\llbracket c \rrbracket$ by induction on the syntax, whereas the [reference](#) for what a program does is the operational semantics $\Downarrow$. Nothing so far says the two agree.

Definition 10 (Reachable exit states)

$$\text{Post}\llbracket c \rrbracket S \triangleq \{ \sigma' \mid \exists \sigma \in S, \langle c, \sigma \rangle \Downarrow \sigma' \}$$

the states in which c can actually *finish*, started anywhere in S .

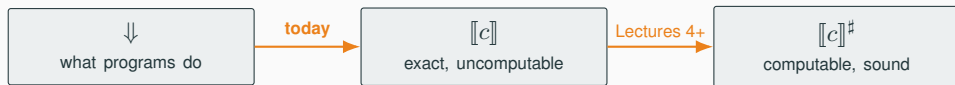
Theorem 11 (Exactness)

For every command c and every $S \subseteq \Sigma$,

$$\text{Post}\llbracket c \rrbracket S = \llbracket c \rrbracket S.$$

We prove the equality by structural induction on c .

Where this theorem fits



Every soundness result from Lecture 4 on is stated *relative to* $\llbracket \cdot \rrbracket$ — the right-hand arrow.

Why the left arrow matters

Collecting semantics is mostly an internal object of the abstract interpretation community. Relating it to the more widely used big-step semantics connects our results to the usual notion of program execution.

Which induction principle?

A first attempt: structural induction on c

It breaks on the loop:

$$\frac{\llbracket b \rrbracket \sigma = \text{tt} \quad \langle c, \sigma \rangle \Downarrow \sigma_1 \quad \langle \text{while } b \text{ do } c, \sigma_1 \rangle \Downarrow \sigma'}{\langle \text{while } b \text{ do } c, \sigma \rangle \Downarrow \sigma'} \text{ WHILE-TRUE}$$

The third premise is about `while  $b$  do  $c$` **itself** — not a strict subterm — so structural induction offers no hypothesis for it.

Factoring the loop into a lemma

A terminating `while b do c` is exactly n iterations of the body, then a failing test. This confines the recursion to the index n .

Lemma 12 (Unrolling)

Define $\sigma \rightarrow_{b,c} \sigma'$ by $\llbracket b \rrbracket \sigma = \text{tt}$ and $\langle c, \sigma \rangle \Downarrow \sigma'$, and let $\rightarrow_{b,c}^n$ denote the n -fold composition of $\rightarrow_{b,c}$. Then

$$\langle \text{while } b \text{ do } c, \sigma \rangle \Downarrow \sigma' \quad \Longleftrightarrow \quad \exists n \geq 0, \quad \sigma \rightarrow_{b,c}^n \sigma' \text{ and } \llbracket b \rrbracket \sigma' = \text{ff}.$$

($\Rightarrow$) by induction on the derivation. WHILE-FALSE: take $n = 0$. WHILE-TRUE: the sub-derivation $\langle \text{while } b \text{ do } c, \sigma_1 \rangle \Downarrow \sigma'$ yields some n with $\sigma_1 \rightarrow_{b,c}^n \sigma'$; prepending $\sigma \rightarrow_{b,c} \sigma_1$ gives $\sigma \rightarrow_{b,c}^{n+1} \sigma'$.

($\Leftarrow$) by induction on n . $n = 0$: $\sigma = \sigma'$, and WHILE-FALSE applies. $n + 1$: from $\sigma \rightarrow_{b,c} \sigma_1 \rightarrow_{b,c}^n \sigma'$ the IH gives $\langle \text{while } b \text{ do } c, \sigma_1 \rangle \Downarrow \sigma'$, and WHILE-TRUE concludes.

The induction statement

We prove the theorem by structural induction on the command c .

Property associated with a command

$$\mathcal{P}(c) \triangleq \forall S \subseteq \Sigma, \quad \text{Post}[[c]] S = [[c]] S.$$

Thus, for a composite command, the induction hypotheses are $\mathcal{P}(c_1)$ and $\mathcal{P}(c_2)$. We now use them to establish the sequencing case.

Exercise — exactness of sequencing

Assume the structural induction hypotheses

$$\forall U, \quad \text{Post}[[c_1]]U = [[c_1]]U \quad \text{and} \quad \text{Post}[[c_2]]U = [[c_2]]U.$$

Prove

$$\text{Post}[[c_1 ; c_2]]S = [[c_1 ; c_2]]S.$$

Where do we use the fact that the induction hypotheses hold for *every* input set U ?

Exercise — exactness of sequencing

Assume the structural induction hypotheses

$$\forall U, \quad \text{Post}[\![c_1]\!]U = \llbracket c_1 \rrbracket U \quad \text{and} \quad \text{Post}[\![c_2]\!]U = \llbracket c_2 \rrbracket U.$$

Prove

$$\text{Post}[\![c_1 ; c_2]\!]S = \llbracket c_1 ; c_2 \rrbracket S.$$

Where do we use the fact that the induction hypotheses hold for *every* input set U ?

Solution

Inversion of the big-step rule for sequencing gives

$$\begin{aligned} \text{Post}[\![c_1 ; c_2]\!]S &= \text{Post}[\![c_2]\!](\text{Post}[\![c_1]\!]S) \\ &= \text{Post}[\![c_2]\!](\llbracket c_1 \rrbracket S) \\ &= \llbracket c_2 \rrbracket (\llbracket c_1 \rrbracket S) = \llbracket c_1 ; c_2 \rrbracket S. \end{aligned}$$

The hypothesis for c_2 is used at $U = \llbracket c_1 \rrbracket S$, not at $U = S$.

Proof — the other structural cases

SKIP — $\text{Post}[\llbracket \text{skip} \rrbracket]S = S = \llbracket \text{skip} \rrbracket S$.

ASSIGN — directly from the big-step rule,

$$\text{Post}[\llbracket x := a \rrbracket]S = \{ \sigma[x \mapsto \llbracket a \rrbracket \sigma] \mid \sigma \in S \} = \llbracket x := a \rrbracket S.$$

IF — by inversion of the two conditional rules, then both induction hypotheses,

$$\begin{aligned} \text{Post}[\llbracket \text{if } b \text{ then } c_1 \text{ else } c_2 \rrbracket]S &= \text{Post}[\llbracket c_1 \rrbracket](\llbracket b \rrbracket S) \cup \text{Post}[\llbracket c_2 \rrbracket](\llbracket \neg b \rrbracket S) \\ &= \llbracket c_1 \rrbracket(\llbracket b \rrbracket S) \cup \llbracket c_2 \rrbracket(\llbracket \neg b \rrbracket S) \\ &= \llbracket \text{if } b \text{ then } c_1 \text{ else } c_2 \rrbracket S. \end{aligned}$$

Proof — the loop: $\text{Post}[\llbracket \text{while } b \text{ do } c \rrbracket S] \subseteq \llbracket \text{while } b \text{ do } c \rrbracket S$

Let $\sigma \in S$ with $\langle \text{while } b \text{ do } c, \sigma \rangle \Downarrow \sigma'$, and put $L = \text{lfp} \Phi_S$. Unfolding $L = \Phi_S(L) = S \cup \llbracket c \rrbracket (\llbracket b \rrbracket L)$ gives at once

$$S \subseteq L \quad \text{and} \quad \llbracket c \rrbracket (\llbracket b \rrbracket L) \subseteq L.$$

By Lemma 12 there is an n with $\sigma \rightarrow_{b,c}^n \sigma'$ and $\llbracket b \rrbracket \sigma' = \text{ff}$.

By induction on n : $\sigma \rightarrow_{b,c}^n \sigma_n \implies \sigma_n \in L$

- $n = 0$: then $\sigma_0 = \sigma \in S \subseteq L$.
- $n + 1$: peel from the right, $\sigma \rightarrow_{b,c}^n \sigma_n \rightarrow_{b,c} \sigma_{n+1}$. The IH gives $\sigma_n \in L$, and $\llbracket b \rrbracket \sigma_n = \text{tt}$ puts $\sigma_n \in \llbracket b \rrbracket L$. From $\langle c, \sigma_n \rangle \Downarrow \sigma_{n+1}$ we get $\sigma_{n+1} \in \text{Post}[\llbracket c \rrbracket (\llbracket b \rrbracket L)] = \llbracket c \rrbracket (\llbracket b \rrbracket L) \subseteq L$, the equality being the [structural IH on the body \$c\$](#) .

So $\sigma' \in L$, and $\llbracket b \rrbracket \sigma' = \text{ff}$ gives $\sigma' \in \llbracket \neg b \rrbracket L = \llbracket \text{while } b \text{ do } c \rrbracket S$. ■

Proof — the loop: $\llbracket \text{while } b \text{ do } c \rrbracket S \subseteq \text{Post} \llbracket \text{while } b \text{ do } c \rrbracket S$

Collect the states that *really* reach the loop head:

$$A \triangleq \{ \sigma' \mid \exists \sigma \in S, \exists n \geq 0, \sigma \rightarrow_{b,c}^n \sigma' \}.$$

Step 1 — $\text{lfp} \Phi_S \subseteq A$, by **Park induction**

Enough to check $\Phi_S(A) \subseteq A$, i.e. two inclusions:

- $S \subseteq A$: take $n = 0$.
- $\llbracket c \rrbracket (\llbracket b \rrbracket A) \subseteq A$: by the structural IH on c at the set $\llbracket b \rrbracket A$, that set lies inside $\text{Post} \llbracket c \rrbracket (\llbracket b \rrbracket A)$, so each element σ'' has $\langle c, \sigma' \rangle \Downarrow \sigma''$ for some $\sigma' \in A$ with $\llbracket b \rrbracket \sigma' = \text{tt}$, i.e. $\sigma' \rightarrow_{b,c} \sigma''$. From the definition of A , we have $\sigma \rightarrow_{b,c}^n \sigma'$ for some $\sigma \in S, n$; appending the step gives $\sigma \rightarrow_{b,c}^{n+1} \sigma''$.

Step 2 — $\llbracket \neg b \rrbracket A \subseteq \text{Post} \llbracket \text{while } b \text{ do } c \rrbracket S$

Such an element is a σ' with $\sigma \rightarrow_{b,c}^n \sigma'$ for some $\sigma \in S, n$, and $\llbracket b \rrbracket \sigma' = \text{ff}$ — precisely the right-hand side of Lemma 12, whose $(\Leftarrow)$ direction returns $\langle \text{while } b \text{ do } c, \sigma \rangle \Downarrow \sigma'$.

By monotonicity of the filter, $\llbracket \text{while } b \text{ do } c \rrbracket S = \llbracket \neg b \rrbracket (\text{lfp} \Phi_S) \subseteq \llbracket \neg b \rrbracket A \subseteq \text{Post} \llbracket \text{while } b \text{ do } c \rrbracket S$. ■

§ 4

Existence of the abstract interpreter

The abstract-interpreter pattern

Let $(L^\sharp, \sqsubseteq, \sqcup, \perp, \top)$ be a lattice with bottom and top elements, and let $A \in L^\sharp$. The abstract interpreter follows this syntax-directed pattern:

$$\begin{aligned}\llbracket \text{skip} \rrbracket^\sharp A &\triangleq A, \\ \llbracket x := a \rrbracket^\sharp A &\triangleq \text{assign}^\sharp(x, a, A), \\ \llbracket c_1 ; c_2 \rrbracket^\sharp A &\triangleq \llbracket c_2 \rrbracket^\sharp (\llbracket c_1 \rrbracket^\sharp A), \\ \llbracket \text{if } b \text{ then } c_1 \text{ else } c_2 \rrbracket^\sharp A &\triangleq \llbracket c_1 \rrbracket^\sharp (\llbracket b \rrbracket^\sharp A) \sqcup \llbracket c_2 \rrbracket^\sharp (\llbracket \neg b \rrbracket^\sharp A), \\ \llbracket \text{while } b \text{ do } c \rrbracket^\sharp A &\triangleq \llbracket \neg b \rrbracket^\sharp (\text{lfp} \Phi_A^\sharp), & \Phi_A^\sharp(X) &\triangleq A \sqcup \llbracket c \rrbracket^\sharp (\llbracket b \rrbracket^\sharp X).\end{aligned}$$

The question left open

Why is $\text{lfp} \Phi_A^\sharp$ meaningful in this setting?

1. Existence

Assume that $L^\sharp$ is a **complete lattice**, and that $\text{assign}^\sharp$ and every $\llbracket b \rrbracket^\sharp$ are monotone. Structural induction then makes $\llbracket c \rrbracket^\sharp$, and therefore $\Phi_A^\sharp$, monotone. Knaster–Tarski gives

$$\text{lfp}\Phi_A^\sharp.$$

This alone gives no guarantee that $\bigsqcup_k (\Phi_A^\sharp)^k(\perp)$ is a fixpoint, or that a finite iterate reaches $\text{lfp}\Phi_A^\sharp$.

2. Finite iteration — an alternative

Alternatively, assume that $\Phi_A^\sharp$ is monotone and that $L^\sharp$ satisfies the **ACC**. The increasing sequence

$$\perp \sqsubseteq \Phi_A^\sharp(\perp) \sqsubseteq (\Phi_A^\sharp)^2(\perp) \sqsubseteq \dots$$

eventually stabilises, and its stationary value is $\text{lfp}\Phi_A^\sharp$. Here **monotonicity**, **$\perp$** , and the **ACC** are enough: neither completeness nor continuity is required.

Other ways to compute, or safely approximate, least fixpoints will be introduced later.

§5

Fixpoint computation on equation systems

Beyond the syntax: systems of equations

Our abstract interpreter for IMP computes least fixpoints locally, one for each loop. It is useful, however, to know how to solve a more general fixpoint problem.

We explain this general construction on the [collecting semantics](#), under the simplifying assumption that its lattice satisfies the [ACC](#).

The equation-system view

Attach one unknown X_p to each [program point](#) p , and one equation per point, saying how X_p is obtained from its predecessors:

$$X_p = F_p(X_1, \dots, X_n).$$

Solving the system replaces the syntax-directed recursion.

The running example as a system

Our loop again, entry set $S = \{0\}$, states identified with $\mathbb{Z}$:

$\triangleright p_0$	entry	$X_0 = S$
while $x \leq 2$ do		$X_1 = X_0 \cup X_3$
$\triangleright p_1$	loop head	$X_2 = X_1 \cap \{x \leq 2\}$
$\triangleright p_2$	test true	$X_3 = \llbracket x := x+1 \rrbracket X_2$
$x := x+1$		
$\triangleright p_3$	after body	$X_4 = X_1 \cap \{x > 2\}$
done		
$\triangleright p_4$	exit	

Least solution

$$X_0 = \{0\}, \quad X_1 = \{0, 1, 2, 3\}, \quad X_2 = \{0, 1, 2\}, \quad X_3 = \{1, 2, 3\}, \quad X_4 = \{3\}.$$

Same head and exit as in Lecture 2 — and only X_1 is genuinely recursive.

The system as a single least fixpoint

Stack the unknowns: $\vec{X} = (X_1, \dots, X_n) \in L^n$, ordered **componentwise**. If L is a complete lattice then so is L^n (with $\perp^n$ and componentwise $\sqcup$).

The system becomes one equation $\vec{X} = F(\vec{X})$ with $F = (F_1, \dots, F_n) : L^n \rightarrow L^n$, monotone as soon as each F_p is. Knaster–Tarski applies: the system has a **least solution** $\text{lfp} F$.

Naive strategy: round-robin (Jacobi) iteration

$\vec{X}^0 = \perp^n$, $\vec{X}^{k+1} = F(\vec{X}^k)$ — recompute *every* component at each round.

Round-robin, round by round

Start from $\vec{X}^0 = \perp^5$. Each column is one complete Jacobi round, in which all five equations are recomputed; – means unchanged.

$$X_0 = S, \quad X_1 = X_0 \cup X_3, \quad X_2 = X_1 \cap \{x \leq 2\}, \quad X_3 = \llbracket x := x+1 \rrbracket X_2, \quad X_4 = X_1 \cap \{x > 2\}.$$

k	0	1	2	3	4	5	6	7	8	9	10	11	12
X_0	$\emptyset$	$\{0\}$	–	–	–	–	–	–	–	–	–	–	–
X_1	$\emptyset$	–	$\{0\}$	–	–	$\{0, 1\}$	–	–	$\{0, 1, 2\}$	–	–	$\{0, 1, 2, 3\}$	–
X_2	$\emptyset$	–	–	$\{0\}$	–	–	$\{0, 1\}$	–	–	$\{0, 1, 2\}$	–	–	–
X_3	$\emptyset$	–	–	–	$\{1\}$	–	–	$\{1, 2\}$	–	–	$\{1, 2, 3\}$	–	–
X_4	$\emptyset$	–	–	–	–	–	–	–	–	–	–	–	$\{3\}$

Wasted work

The equation for X_4 is evaluated at every round, but its value changes only at round 12, once 3 has reached X_1 .

Do not update all components at once — update **one at a time**:

$$\vec{X}_{i_k}^{k+1} = \vec{X}_{i_k}^k \sqcup F_{i_k}(\vec{X}^k), \quad \vec{X}_j^{k+1} = \vec{X}_j^k \quad (j \neq i_k),$$

starting from $\perp^n$, choosing the index i_k *however we like* — provided the choice is **fair**: no pending component is ignored forever.

Theorem 13 (Chaotic iteration)

*If F is monotone and the choice of components is fair, the sequence is increasing and stays $\sqsubseteq \text{lfp} F$. If L satisfies the **ascending chain condition**, it stabilises after finitely many steps on exactly $\text{lfp} F$.*

One chaotic iteration strategy

Each column updates only the selected equation; — means unchanged.

k selected	0	1	2	3	4	5	6	7	8	9	10	11	12
	—	X_0	X_1	X_2	X_3	X_1	X_2	X_3	X_1	X_2	X_3	X_1	X_4
X_0	$\emptyset$	$\{0\}$	—	—	—	—	—	—	—	—	—	—	—
X_1	$\emptyset$	—	$\{0\}$	—	—	$\{0, 1\}$	—	—	$\{0, 1, 2\}$	—	—	$\{0, 1, 2, 3\}$	—
X_2	$\emptyset$	—	—	$\{0\}$	—	—	$\{0, 1\}$	—	—	$\{0, 1, 2\}$	—	—	—
X_3	$\emptyset$	—	—	—	$\{1\}$	—	—	$\{1, 2\}$	—	—	$\{1, 2, 3\}$	—	—
X_4	$\emptyset$	—	—	—	—	—	—	—	—	—	—	—	$\{3\}$

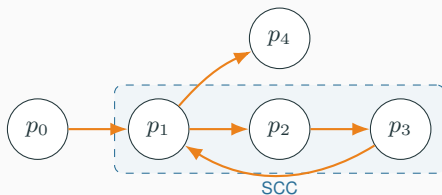
Same result, less work

This strategy reaches the least solution after 12 component updates; Jacobi used $12 \times 5 = 60$ updates. The order changes the cost, not the result.

A static strategy from dependencies

Draw an edge $p \rightarrow q$ when the equation for X_q reads X_p ; equivalently, $q \in \text{dep}(p)$.

$$X_0 = S, \quad X_1 = X_0 \cup X_3, \quad X_2 = X_1 \cap \{x \leq 2\}, \quad X_3 = \llbracket x := x+1 \rrbracket X_2, \quad X_4 = X_1 \cap \{x > 2\}.$$



From graph to strategy

1. compute the strongly connected components;
2. order the resulting acyclic graph;
3. iterate locally inside each recursive component.

Bourdoncle's strategy. The algorithm computes a **weak topological order**; here, one possible order is $p_0 (p_1 p_2 p_3) p_4$, with p_1 as the recursive head. This is exactly the order used in the preceding example, with $p_i = X_i$.

F. Bourdoncle. *Efficient Chaotic Iteration Strategies with Widenings*. FMPA, 1993.

Generating the strategy dynamically: worklist

The worklist W contains the equations whose right-hand side may have changed, where $\text{dep}(p) \triangleq \{q \mid F_q(X) \text{ reads } X[p]\}$.

Worklist

```
for each point  $p$ :  $X[p] := \perp$   
 $W := \{\text{all points}\}$   
while  $W \neq \emptyset$ :  
  pick  $p \in W$ ;  $W := W \setminus \{p\}$   
   $\text{new} := F_p(X)$   
  if  $\text{new} \not\sqsubseteq X[p]$ :  
     $X[p] := X[p] \sqcup \text{new}$   
     $W := W \cup \text{dep}(p)$   
return  $X$ 
```

Our example

$X_0 = S$	$\text{dep}(p_0) = \{p_1\},$
$X_1 = X_0 \cup X_3$	$\text{dep}(p_1) = \{p_2, p_4\},$
$X_2 = X_1 \cap \{x \leq 2\}$	$\text{dep}(p_2) = \{p_3\},$
$X_3 = \llbracket x := x+1 \rrbracket X_2$	$\text{dep}(p_3) = \{p_1\},$
$X_4 = X_1 \cap \{x > 2\}$	$\text{dep}(p_4) = \emptyset.$

The successive choices of p form the chaotic strategy. Whenever $X[p]$ grows, only the equations that read it are added to W : no affected equation is forgotten.

Exercise — execute the worklist

All X_p start at $\emptyset$, and all points are initially in W . Complete the last two columns for the given sequence of picks.

	pick	recomputed	W afterwards
1	p_0	$X_0 = \{0\}$	$p_1 \ p_2 \ p_3 \ p_4$
2	p_1		
3	p_2		
4	p_3		
5	p_1		
6	p_2		
7	p_3		
8	p_1		
9	p_2		
10	p_3		
11	p_1		
12	p_2		
13	p_4		

System

$$X_0 = S,$$

$$X_1 = X_0 \cup X_3,$$

$$X_2 = X_1 \cap \{x \leq 2\},$$

$$X_3 = \llbracket x := x+1 \rrbracket X_2,$$

$$X_4 = X_1 \cap \{x > 2\}.$$

Dependencies

$$\text{dep}(p_0) = \{p_1\},$$

$$\text{dep}(p_1) = \{p_2, p_4\},$$

$$\text{dep}(p_2) = \{p_3\},$$

$$\text{dep}(p_3) = \{p_1\},$$

$$\text{dep}(p_4) = \emptyset.$$

Exercise — execute the worklist

All X_p start at $\emptyset$, and all points are initially in W . Complete the last two columns for the given sequence of picks.

	pick	recomputed	W afterwards
1	p_0	$X_0 = \{0\}$	$p_1 \ p_2 \ p_3 \ p_4$
2	p_1	$X_1 = \{0\}$	$p_2 \ p_3 \ p_4$
3	p_2	$X_2 = \{0\}$	$p_3 \ p_4$
4	p_3	$X_3 = \{1\}$	$p_1 \quad p_4$
5	p_1	$X_1 = \{0, 1\}$	$p_2 \quad p_4$
6	p_2	$X_2 = \{0, 1\}$	$p_3 \ p_4$
7	p_3	$X_3 = \{1, 2\}$	$p_1 \quad p_4$
8	p_1	$X_1 = \{0, 1, 2\}$	$p_2 \quad p_4$
9	p_2	$X_2 = \{0, 1, 2\}$	$p_3 \ p_4$
10	p_3	$X_3 = \{1, 2, 3\}$	$p_1 \quad p_4$
11	p_1	$X_1 = \{0, 1, 2, 3\}$	$p_2 \quad p_4$
12	p_2	unchanged	p_4
13	p_4	$X_4 = \{3\}$	$\emptyset$

At p_1 we recover the Kleene iterates $\{0\} \subset \{0, 1\} \subset \{0, 1, 2\} \subset \{0, 1, 2, 3\}$. Step 12 makes no change and therefore schedules no successor; p_4 is picked only once, at the end.

System

$$X_0 = S,$$

$$X_1 = X_0 \cup X_3,$$

$$X_2 = X_1 \cap \{x \leq 2\},$$

$$X_3 = \llbracket x := x+1 \rrbracket X_2,$$

$$X_4 = X_1 \cap \{x > 2\}.$$

Dependencies

$$\text{dep}(p_0) = \{p_1\},$$

$$\text{dep}(p_1) = \{p_2, p_4\},$$

$$\text{dep}(p_2) = \{p_3\},$$

$$\text{dep}(p_3) = \{p_1\},$$

$$\text{dep}(p_4) = \emptyset.$$

Correctness: two invariants

- $X \sqsubseteq \text{lfp}F$: true at $\perp^n$, and preserved because $\text{new} = F_p(X) \sqsubseteq F_p(\text{lfp}F) = (\text{lfp}F)_p$ by monotonicity, hence $X[p] \sqcup \text{new} \sqsubseteq (\text{lfp}F)_p$.
- $q \notin W \implies F_q(X) \sqsubseteq X[q]$. Initially this is vacuous. When p leaves W , its equation has just been checked. For a point q already outside W , $X[q]$ is unchanged, and $F_q(X)$ can change only if $q \in \text{dep}(p)$; then q is put back into W .

On exit, $W = \emptyset$, so the second invariant gives $F(X) \sqsubseteq X$. Park gives $\text{lfp}F \sqsubseteq X$; hence $X = \text{lfp}F$ by the first invariant.

Termination

Order configurations lexicographically by

$$(X', m') \prec (X, m) \iff X \sqsubset X' \text{ or } (X' = X \text{ and } m' < m).$$

Each iteration strictly decreases $(X, |W|)$: either X grows, or X is unchanged and W loses one element. Since there are finitely many points, the ACC on L gives the ACC on L^n ; hence $\prec$ is well-founded.

Orders, lattices, fixpoints (§1–§2)

- B. A. Davey, H. A. Priestley. *Introduction to Lattices and Order*. CUP, 2nd ed. 2002 — the standard reference.

Worklist algorithms and iteration strategies (§4)

- F. Nielson, H. R. Nielson, C. Hankin. *Principles of Program Analysis*. Springer, 1999, §6.1 “Worklist Algorithms.”
- F. Bourdoncle. *Efficient chaotic iteration strategies with widenings*. FMPA 1993 — weak topological order.